UCS503 – Software Engineering

Campus Marketplace

A Verified-Student Platform for Buying, Borrowing, Selling and Lending
with ML-Assisted Pricing and Fraud Detection

Submitted by:

1024031047 Shweta Agarwal
1024031049 Shrasthi Singh Rathore
1024031021 Raksha Pandey

Group: 3C73 BE Third Year, CSE

Submitted to:

Dr. Johnsy Bansal

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Abstract

Students on a residential campus constantly need second-hand textbooks, cycles, calculators, lab equipment and hostel essentials, and just as constantly have such items to pass on. General-purpose classified platforms are poorly suited to this need: they are open to anonymous strangers, offer no verification that a seller belongs to the institution, give no help with fair pricing, and do not support lending with due dates and returns. Campus Marketplace is a web application that addresses these gaps for a single campus community.

The system lets students with verified college e-mail identities buy, borrow, sell and lend items. It is built as a React single-page application, an Express.js REST API backed by MongoDB, and a separate Python FastAPI service that hosts two machine-learning models: a Random Forest regressor that recommends a fair price range and an Isolation Forest that scores the fraud risk of every new listing. Further modules provide e-mail OTP and Google Sign-In verification, a loan life-cycle with extensions and return confirmation, a payment flow with server-side signature verification, privacy-preserving nearby discovery based on geohashes, a trust score for every user, and an administrator moderation queue.

This report documents the requirements, the development methodology, the architecture, and the implementation of the system, supported by five diagrams (use case, SDLC, sequence, activity and class). On the held-out test split of the synthetic seed data, the price model reaches $R^2 = 0.623$ and a mean absolute error of Rs. 992; this figure is reported transparently as a baseline, since the models have not yet been trained on real campus transactions.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background and Context . . . 1

1.1.1 Problem Statement . . . 1

1.2 Project Objectives . . . 1

1.3 Scope and Limitations . . . 2

1.4 Organisation of the Report . . . 2

2 System Requirements **3**

2.1 Stakeholders and User Roles . . . 3

2.2 Functional Requirements . . . 3

2.3 Non-Functional Requirements . . . 3

2.4 Hardware and Software Requirements . . . 3

2.5 Constraints and Assumptions . . . 3

3 Architecture and Design **5**

3.1 Development Methodology (SDLC) . . . 5

3.2 Use Case Model . . . 6

3.3 Sequence Model . . . 6

3.4 Activity Model . . . 7

3.5 Class Model . . . 7

4 Implementation **10**

4.1 Technology Stack . . . 10

4.2 Frontend . . . 10

4.3 Backend REST API . . . 11

4.4 Authentication and Verified Access . . . 11

4.5 Listings, Fair-Price Assistance and Fraud Screening . . . 12

4.5.1 Creating a listing . . . 12

4.5.2 Price recommendation . . . 12

4.5.3 Fraud scoring . . . 12

4.6 Loan and Transaction Life Cycle 12
4.7 Payments 13
4.8 Location Privacy 13
4.9 Trust Score and Moderation 13
4.10 ML Service 13
4.11 Testing and Results 14
4.11.1 Automated and manual testing 14
4.11.2 Model performance 14
4.11.3 Analysis 14
4.12 Challenges and Resolutions 15

5 Conclusion **16**
5.1 Summary of Work 16
5.2 Key Findings 16
5.3 Limitations 16
5.4 Future Work and Recommendations 17
5.5 Final Remarks 17

Bibliography **18**

List of Figures

3.1 SDLC diagram: iterative and incremental model used for Campus Marketplace. . . . 5
3.2 Use case diagram of Campus Marketplace. 6
3.3 Sequence diagram: publishing a listing with price recommendation and fraud screening. 7
3.4 Activity diagram: sign-in, listing, moderation and transaction flow. 8
3.5 Class diagram of the persistent domain model. 9

List of Tables

2.1 Stakeholders and their interaction with the system. 3
2.2 Functional requirements. 4
2.3 Non-functional requirements. 4
2.4 Development and runtime environment. 4

4.1 Technology stack by layer. 10
4.2 REST API modules (all under `/api`). 11
4.3 Condition factors used by the fair-price formula. 12
4.4 Behaviours verified by the automated API tests. 14
4.5 Random Forest price model: validation and test metrics ($n_{\text{train}} = 196$, $n_{\text{val}} = 42$, $n_{\text{test}} = 42$). 14
4.6 Main problems met during development and how they were resolved. 15

Chapter 1

Introduction

1.1 Background and Context

A university campus is a small, dense economy. Every semester, first-year students buy the same textbooks, drafters and lab coats that graduating students are about to discard; hostel residents need kettles, mattresses and cycles for a few months only; and short-term needs such as a scientific calculator for one examination are better served by borrowing than by purchasing. Today this exchange happens through scattered group chats and notice boards, which are unsearchable, unstructured and unaccountable.

Public classified websites do not solve the problem. They are open to anyone, so a buyer cannot tell whether the seller is a genuine fellow student; they provide no guidance on what a used item is worth; and they have no notion of lending, where an item changes hands for a limited period and must be returned. A purpose-built, campus-exclusive marketplace can address each of these shortcomings, and doing so is a good vehicle for applying the full software engineering life cycle: requirements, design, implementation and evaluation of a multi-tier, data-driven system.

1.1.1 Problem Statement

Design and implement a web platform in which only verified members of a campus can list items for sale or for lending, discover items nearby, communicate with each other, complete purchases or loans with an auditable transaction history, and build a measurable reputation, while the system itself assists users with fair pricing and protects the community against fraudulent or duplicated listings.

1.2 Project Objectives

1. **Verified access:** restrict the marketplace to users whose college e-mail identity has been verified (OTP or Google Sign-In), and reject unverified sessions on every marketplace endpoint.
2. **Complete trading workflow:** support the four intents buy, borrow, sell and lend, including a loan life-cycle with due dates, extensions, overdue detection and return confirmation.
3. **ML-assisted pricing:** give every seller an explainable fair price range produced by a trained regression model, with the final price remaining the seller's decision.
4. **Fraud screening:** score every new listing for anomalous behaviour and route high-risk listings to an administrator before they become visible.

5. **Trust and privacy:** compute a trust score for each user, keep exact coordinates private, and reveal meeting points only after both parties agree.
6. **Maintainable architecture:** separate the user interface, the business API and the ML service so that each can be developed, tested and deployed independently, and verify the API with an automated test suite.

1.3 Scope and Limitations

The system targets a single campus whose allowed e-mail domains are configured by the administrator. Online payment runs in a clearly labelled development mode unless Razorpay test credentials are supplied. The machine-learning models are trained on a synthetic seed data set and are therefore a demonstrator of the architecture rather than a production-grade valuation engine. These limitations are discussed again in Chapter 5.

1.4 Organisation of the Report

Chapter 2 lists the stakeholders and the functional and non-functional requirements. Chapter 3 presents the development methodology and the design through five diagrams. Chapter 4 describes the implementation and evaluation, and Chapter 5 concludes with findings, limitations and future work.

Chapter 2

System Requirements

2.1 Stakeholders and User Roles

Table 2.1. Stakeholders and their interaction with the system.

Role	Description
Student	A verified campus member. The same account can act as buyer, borrower, seller or lender, so a single role covers all four intents.
Administrator	Reviews held listings in the fraud-alert queue and approves or rejects them.
ML service	Internal system actor that returns price recommendations and fraud scores to the API over HTTP.
Payment gateway	External actor (Razorpay, or a signed development gateway) that creates orders and confirms payments.
Identity provider	Google OAuth 2.0 and an SMTP server used for verification.

2.2 Functional Requirements

2.3 Non-Functional Requirements

2.4 Hardware and Software Requirements

2.5 Constraints and Assumptions

All users hold an e-mail address in an allowed college domain or an allowed Google account. Trading takes place physically on campus, so the system arranges but does not deliver exchanges. No map provider key is required; location relies on the browser Geolocation API and geohashing only.

Table 2.2. Functional requirements.

ID	Requirement
FR1	A user shall register with a college e-mail, verify it with a one-time password, and log in; Gmail and allowed Google accounts shall use Google Sign-In.
FR2	The system shall reject any marketplace request from an unverified user, and any administrator request from a non-administrator.
FR3	A student shall create, edit, view and remove listings for sale or lending, with category, condition, brand, age, original price, description and images.
FR4	The system shall return a fair-price range, a recommended price, a confidence value and an explanation for a new listing.
FR5	The system shall assign each new listing a fraud-risk score and level (low, medium, high) and hold high-risk listings for review.
FR6	A student shall search and filter listings and keep a watchlist.
FR7	A student shall chat with a listing owner through stored messages.
FR8	A student shall request to buy or borrow; the owner shall approve or reject, set or change the due date, and approve extensions.
FR9	The system shall track loan status (requested, active, due soon, overdue, return requested, returned, completed) and notify users.
FR10	Sales shall be settled through a payment order whose signature is verified on the server before the sale is marked paid.
FR11	Users shall be able to opt in to nearby discovery; exact coordinates shall never be returned by listing APIs.
FR12	The system shall maintain a trust score from completed transactions, on-time returns, reviews and confirmed reports.
FR13	An administrator shall view fraud alerts and approve or reject held listings.

Table 2.3. Non-functional requirements.

Attribute	Requirement
Security	Passwords hashed with bcrypt; OTPs stored only as salted hashes with expiry, attempt and resend limits; stateless JWT sessions; rate limiting on authentication, OTP and general API traffic.
Privacy	Exact latitude and longitude are never exposed; approximate distance only, and only for users who opted in.
Availability	The API shall start without an installed MongoDB (in-memory fallback) and shall keep working when the ML service is down.
Maintainability	Three independently runnable components with environment-based configuration and a documented REST interface.
Usability	Responsive interface; clear labels for development-mode payments and for AI estimates that are not guaranteed market values.
Testability	Automated API tests for authentication, listings, location, loans, payments, AI endpoints and trust score.

Table 2.4. Development and runtime environment.

Item	Requirement
Client	Any modern browser with JavaScript; Geolocation API for optional nearby discovery.
Server	Node.js runtime with npm; MongoDB (local, Atlas, or the bundled in-memory server); Python 3 for the ML service.
Libraries	React, React Router, Tailwind CSS, Vite; Express, Mongoose, jsonwebtoken, bcryptjs, nodemailer, Razorpay SDK, Google Auth Library; FastAPI, scikit-learn, pandas, NumPy, joblib.
Hardware	A commodity laptop is sufficient for development and demonstration.

Chapter 3

Architecture and Design

This chapter presents the development methodology and the design of Campus Marketplace. All diagrams are drawn natively in TikZ and use a monochrome palette: black lines, white fills, and a light gray for emphasis.

3.1 Development Methodology (SDLC)

The project followed an iterative and incremental life cycle [4]. Rather than building the whole system in one pass, the work was divided into five increments. Each increment was taken through the same five phases (requirements, design, implementation, testing, and integration review), and the feedback from the review fed directly into the requirements of the next increment. Figure 3.1 shows the cycle and the increments delivered. This model suited the project because the user interface existed first as a prototype on mock data, and the backend, identity, marketplace logic and machine-learning components were added in successive, independently testable steps.

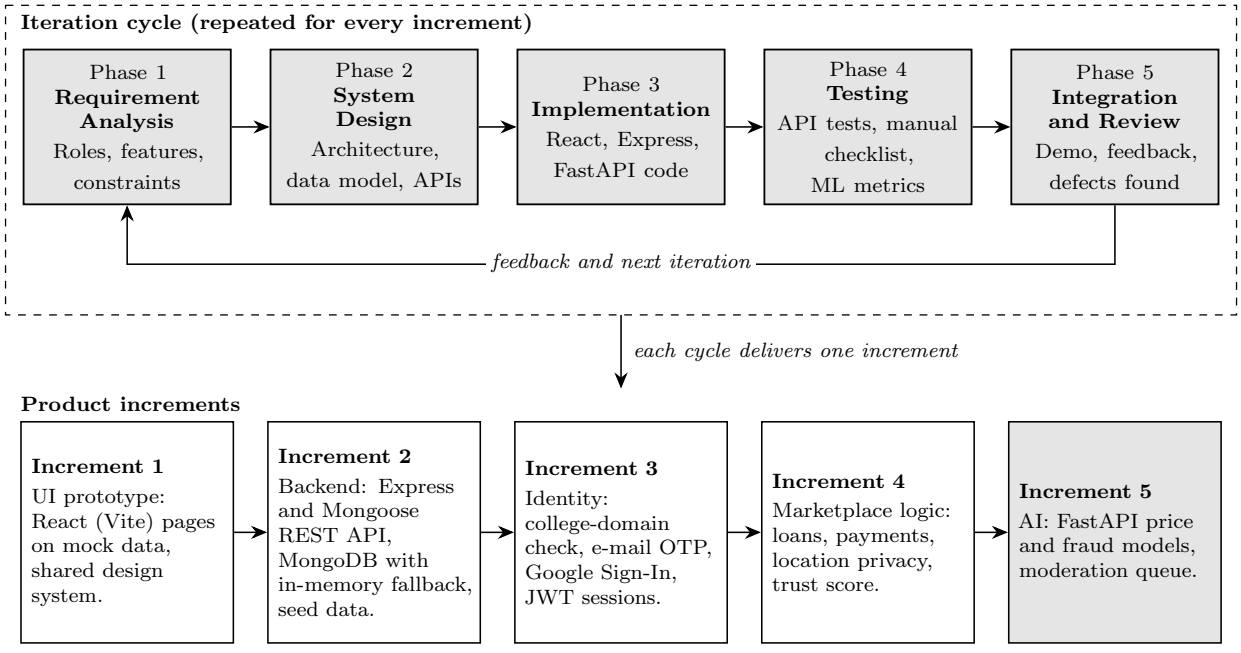


Figure 3.1. SDLC diagram: iterative and incremental model used for Campus Marketplace.

3.2 Use Case Model

Figure 3.2 shows the use cases inside the system boundary. A single Student actor covers buyer, borrower, seller and lender because the same account performs all four roles. The administrator moderates held listings; the ML service and the payment gateway are secondary actors that the system calls. Price prediction and fraud scoring *extends* listing creation, and making a payment *includes* signature verification.

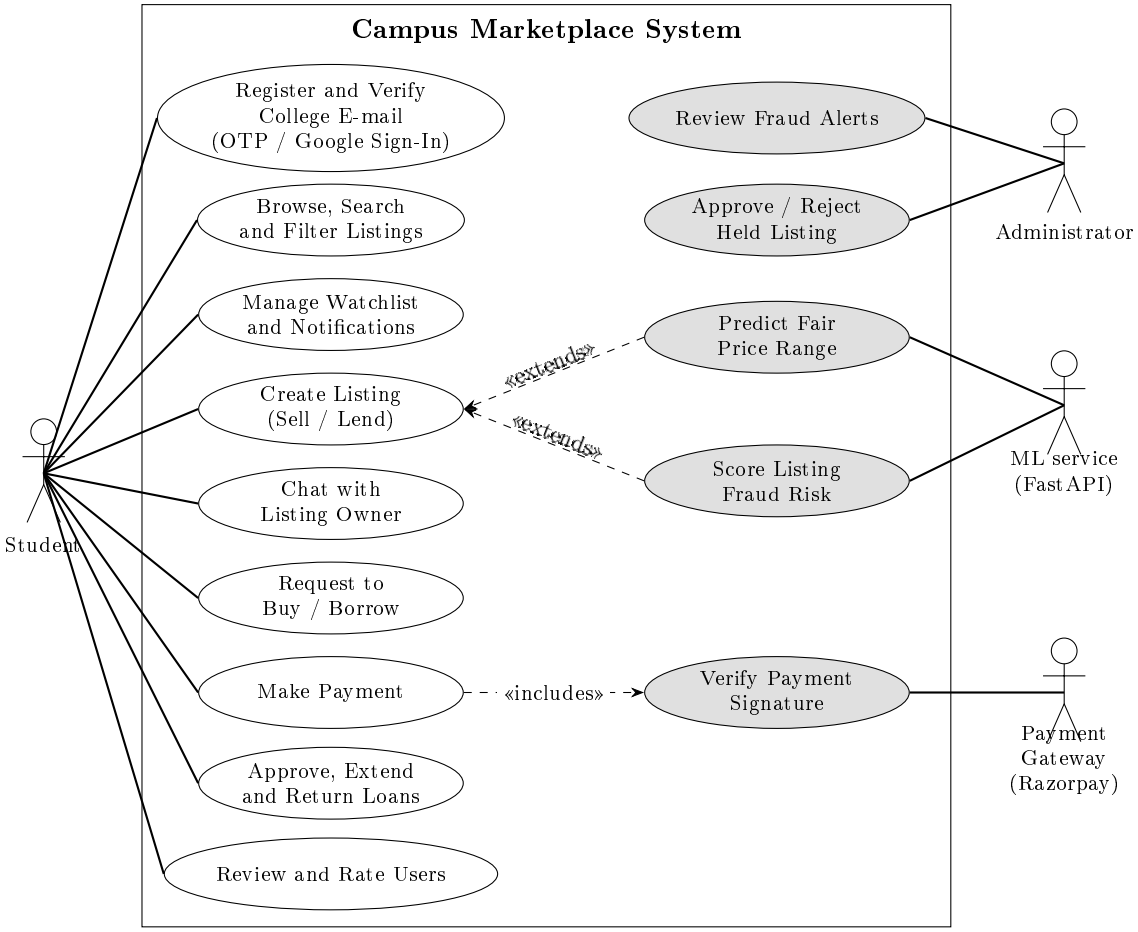


Figure 3.2. Use case diagram of Campus Marketplace.

3.3 Sequence Model

The system is organised as three independently runnable tiers: a React single-page application that talks only to the REST API through a central client module, an Express.js API that applies CORS, rate limiting and JWT authentication before requests reach the route modules and business services, and a separate FastAPI service that hosts the price and fraud models. Persistent data lives in MongoDB, and identity and payment providers are external services. Keeping the ML stack in its own service means it can be trained, replaced or restarted without touching the API, and the API degrades gracefully when that service is unreachable.

Figure 3.3 shows how these parts interact over time when a seller publishes a listing. The seller

submits the details through the interface, which sends a request to the API. The API asks the ML service for a fair-price range, then for a fraud-risk score, saves the listing in MongoDB, and returns the result to the interface for display.

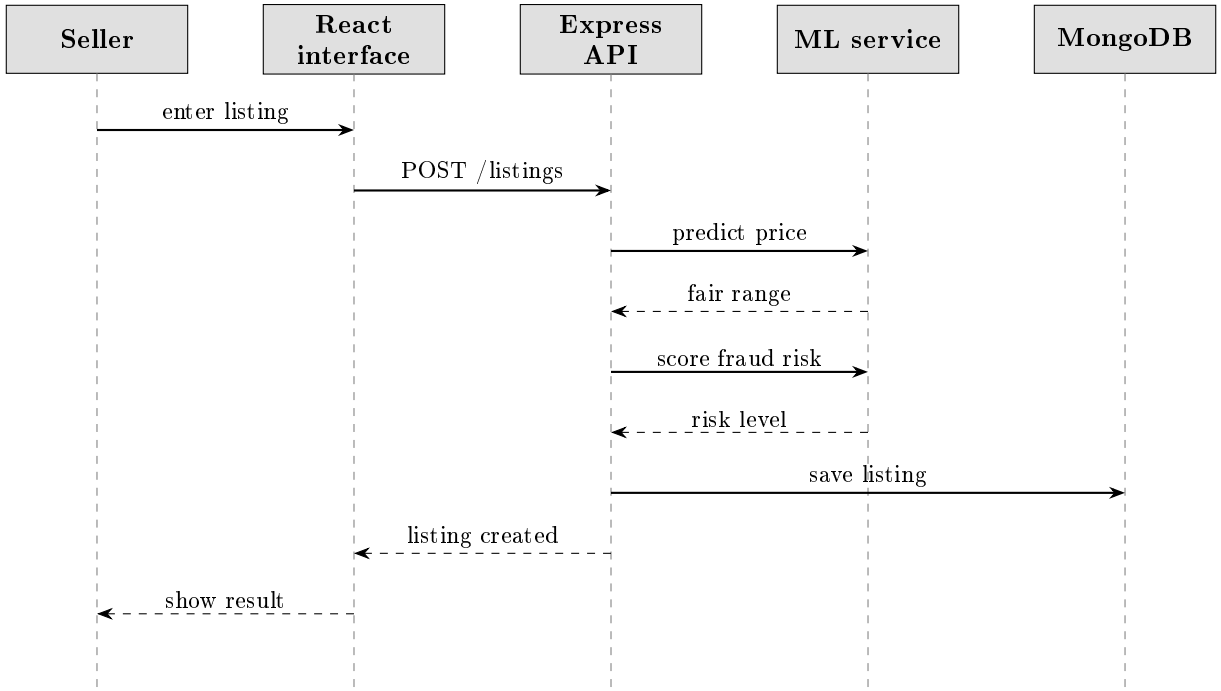


Figure 3.3. Sequence diagram: publishing a listing with price recommendation and fraud screening.

3.4 Activity Model

Figure 3.4 traces the end-to-end flow from sign-in to a completed transaction. A user first proves a verified college identity. A seller then enters listing details, the ML service returns a fair-price range, and after the seller fixes an asking price the system computes a fraud-risk score; a high-risk listing is held and goes live only if an administrator approves it. A buyer later requests the item; if the owner approves, a sale proceeds through payment-signature verification, while a loan proceeds through agreement on a meeting point and due date and ends with a lender-confirmed return. Rejections terminate the flow at activity final nodes.

3.5 Class Model

Figure 3.5 shows the principal persistent classes. Each class maps to a Mongoose schema and a MongoDB collection; the attribute lists are abbreviated to the fields central to the design. User is the hub: it owns listings, takes part in transactions, sends and receives messages, and receives reviews. A Listing is the subject of transactions and may be screened in the Moderation queue. A Transaction may be settled by one Payment. Types such as Number and ObjectId follow the Mongoose schema types.

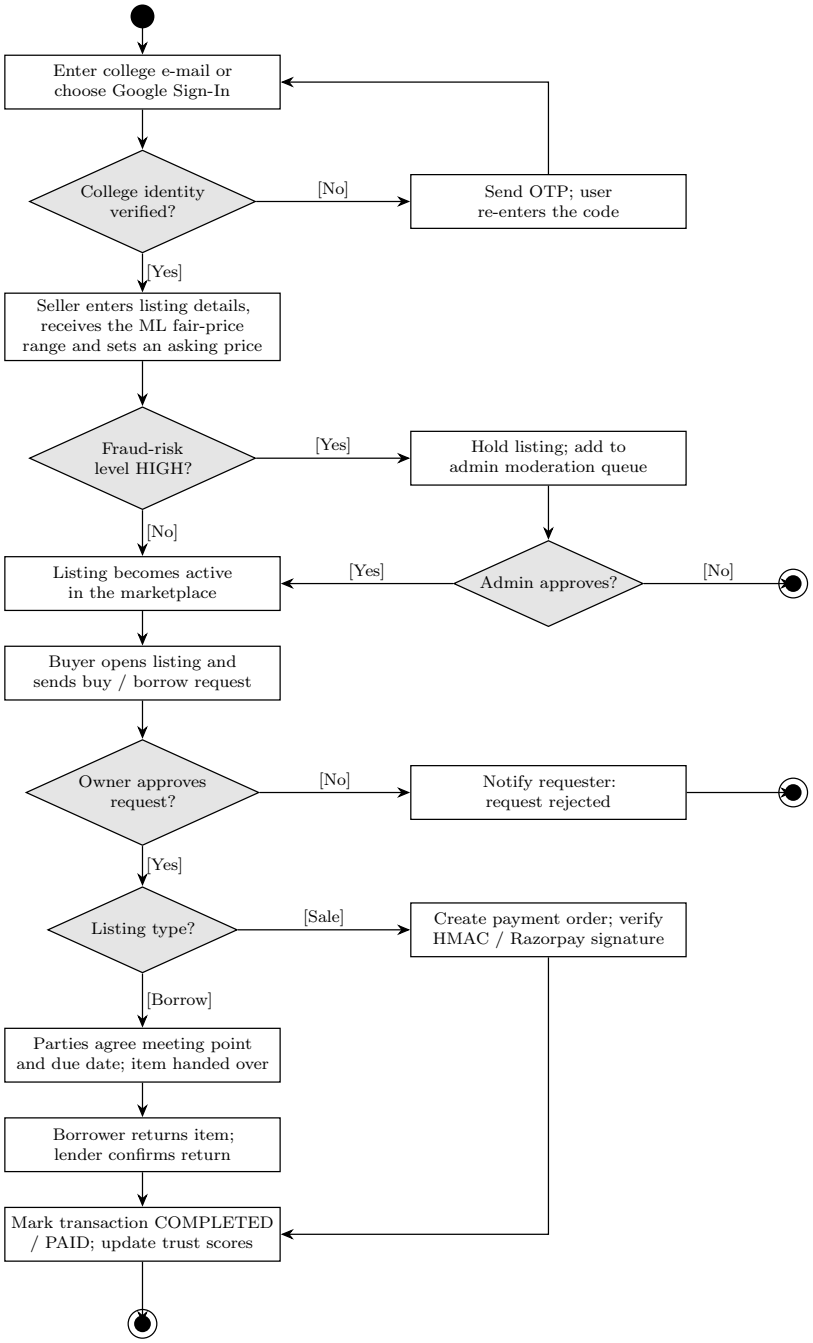


Figure 3.4. Activity diagram: sign-in, listing, moderation and transaction flow.

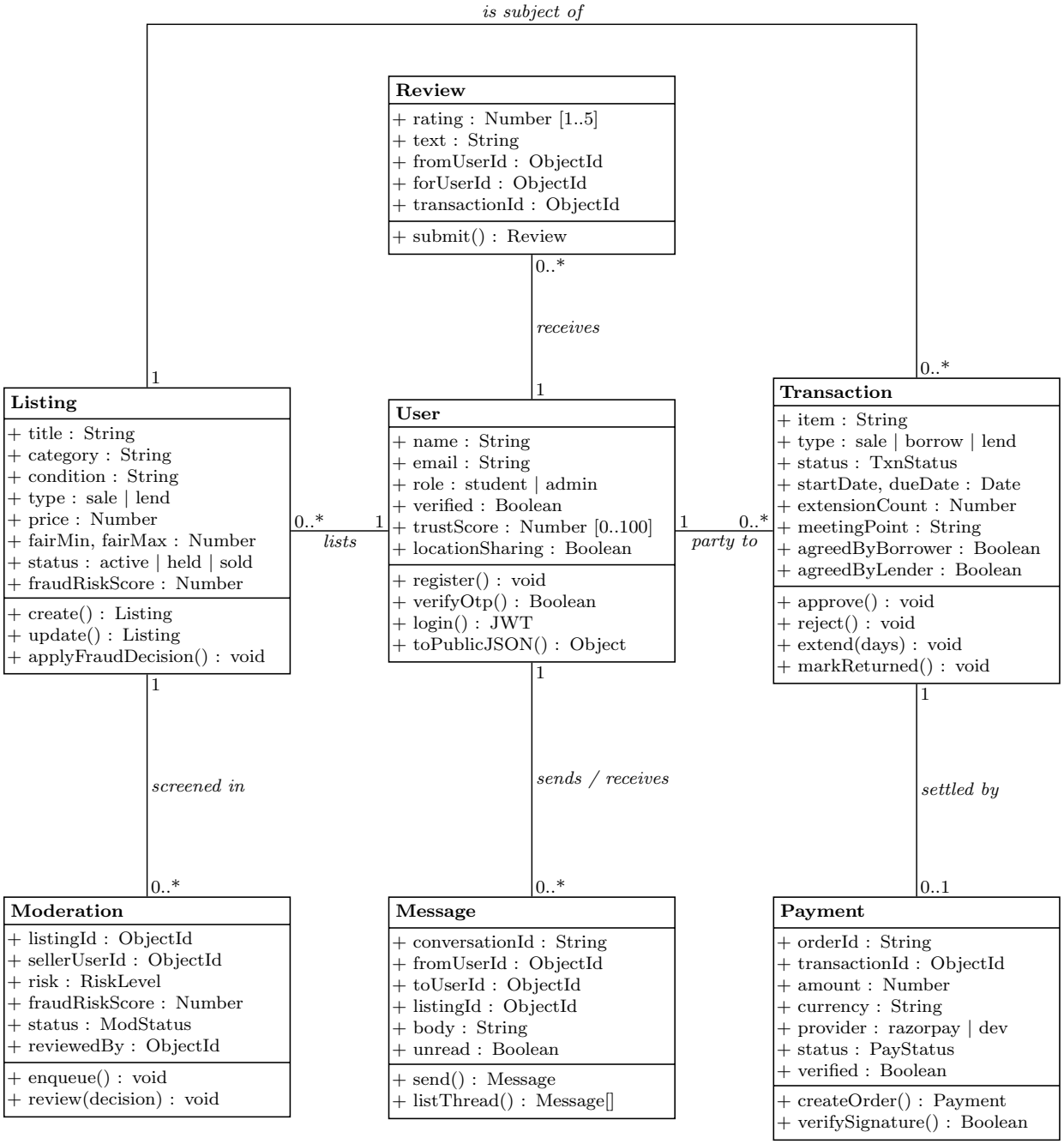


Figure 3.5. Class diagram of the persistent domain model.

Chapter 4

Implementation

4.1 Technology Stack

Table 4.1. Technology stack by layer.

Layer	Technology	Purpose
Frontend	React 19, React Router 7, Vite, Tailwind CSS [12]	Single-page UI, protected routes, filters, favourites, dashboards.
API	Node.js, Express 4 [13], Mongoose	REST endpoints, validation, business rules.
Database	MongoDB [14]	Persistence for users, listings, loans, messages, payments and reviews.
Security	jsonwebtoken, bcryptjs, express-rate-limit	Sessions [8], password hashing, throttling.
Identity	Google Auth Library, nodemailer	OAuth 2.0 sign-in [9] and OTP e-mail.
Payments	Razorpay SDK, HMAC-SHA256 [10]	Order creation and signature verification.
ML service	Python, FastAPI [15], scikit-learn [3], pandas, joblib	Price regression and fraud anomaly scoring.
Quality	oxlint, Node test runner	Static analysis and API tests.

The repository is organised so that each tier can be run on its own: `src/` holds the React pages and components; `server/models`, `server/routes`, `server/middleware` and `server/utils` hold the Mongoose schemas, REST routers, authentication and rate-limit middleware, and business services; `server/seed.js` creates demonstration data; and `ml-service/` contains the FastAPI application, the training scripts, the seed data set and the serialised models.

4.2 Frontend

The interface is a React single-page application with ten routes: sign-in and the Google callback, a dashboard, Buy/Borrow, Sell/Lend, My Listings, Messages, Transactions, Notifications, Profile and the administrator moderation page. A Protected wrapper redirects unauthenticated visitors to the sign-in page. All network access goes through one module, `src/api.js`, which attaches the JWT to each request. During development Vite proxies `/api` to the Express server, which removed the cross-origin problems that had initially appeared between the two ports.

Listings originally referenced Lucide icon components, which cannot be stored in a database.

The schema therefore stores a string `iconKey` and the client maps it back to a component when a listing is rendered (`hydrateListing`).

4.3 Backend REST API

The Express application applies CORS, JSON parsing with a body-size limit, a global rate limiter and an optional-authentication middleware before mounting the routers summarised in Table 4.2. Every marketplace route is wrapped by `requireAuth` and `requireVerified`; administrator routes additionally use `requireAdmin`. A health endpoint reports the database state, the payment mode and the ML service address.

Table 4.2. REST API modules (all under `/api`).

Module	Prefix	Operations
Auth	`/auth`	register, resend-otp, verify, login, logout, me, google, google/start, google/callback
Listings	`/listings`	search and list, mine, read, create, update, delete
Transactions	`/transactions`	request, approve, reject, due-date, return, extend, meeting
Payments	`/payments`	create-order, verify, dev-complete, read
Messages	`/messages`	list conversations, read thread, send
Watchlist	`/watchlist`	list, add, remove
Notifications	`/notifications`	list, mark as read
Location	`/location`	preferences, update position, nearby
Users, reviews	`/users`	profile, trust-score, reviews
AI	`/ai`	price-predict, fraud-score
Admin	`/admin`	fraud-alerts, review of a held listing

4.4 Authentication and Verified Access

Registration accepts only addresses in the configured college domains. The server generates a one-time password, stores only a salted hash of it together with an expiry time, an attempt counter and a resend counter, and delivers it by SMTP (or returns it in the response when a development flag is set). Passwords are hashed with bcrypt. Verification failures, expired codes and rapid resends are limited by dedicated rate limiters.

Users with a Google account follow the OAuth 2.0 authorization-code flow [9]: the API redirects to Google's account picker, exchanges the returned code, checks that the e-mail address is verified by Google and belongs to an allowed domain, and then issues the same JWT [8] that password login produces. Because the token carries only the user identity, the API remains stateless; the verified flag and role are re-read from the database on every request, so revoking verification takes effect immediately.

4.5 Listings, Fair-Price Assistance and Fraud Screening

4.5.1 Creating a listing

When a seller publishes a listing, the API validates the input, requests a price recommendation, stores the recommended range on the listing, and then assesses fraud risk. If the ML service cannot be reached, the listing is still created and the interface shows that the AI recommendation is temporarily unavailable.

4.5.2 Price recommendation

The ML service first computes a depreciation-style formula price from the original price, the condition, the age in months and the listing type, using the condition factors of Table 4.3. It then predicts a price with a Random Forest regressor [1] trained on seven encoded features: original price, age, demand, category, condition, listing type and brand-name length. To keep the output explainable and bounded, the two estimates are combined: the recommended price is a weighted blend that gives the greater weight to the formula price, and when the model output strays too far from the formula price the formula price is used on its own. The fair range is centred on the recommended price, and its width and the reported confidence are derived from the spread of the relative validation residuals, so that a less reliable model yields a wider range and a lower confidence. The seller can ignore the suggestion; the asking price is always the seller's choice.

Table 4.3. Condition factors used by the fair-price formula.

Condition	New	Like New	Good	Fair
Factor	0.92	0.78	0.62	0.45

4.5.3 Fraud scoring

For each new listing the API assembles thirteen features: price, original price, the ratio of the price to the category mean, the ratio of the price to the original price, title length, description length, the number of listings with the same title, the number with the same description hash, the number of listings the seller posted in the last hour, confirmed reports against the seller, transactions in the last 24 hours, account age, and whether the seller is verified. The FastAPI service evaluates them with an Isolation Forest [2], which isolates anomalies with few random splits. The model's decision value is converted into a risk score between 0 and 1, which is then mapped to a risk level: low, medium or high, with high reserved for the most anomalous listings. Listings with level high are given the status *held*, a record is added to the moderation queue, and the listing is hidden from other students until an administrator approves it (Figure 3.4).

4.6 Loan and Transaction Life Cycle

Each transaction document carries a status that moves through a small state machine. A borrow request starts as REQUESTED; when the lender approves it becomes ACTIVE. A utility function derives DUE SOON and OVERDUE from the due date and is applied when loans are listed and refreshed, which also raises notifications. The borrower marks the item RETURN REQUESTED;

the lender's confirmation completes the loan as COMPLETED. Extension requests are stored in an extension history and must be approved by the lender. A purchase instead starts as PAYMENT PENDING and becomes PAID only after the payment is verified. Rejected and cancelled requests end as REJECTED or CANCELLED.

4.7 Payments

Creating a purchase produces a Payment document with an order identifier. With Razorpay keys configured, the server creates a Razorpay order and later validates the checkout signature; without keys, a development gateway signs the order with an HMAC [10] using a server-side secret. In both modes `POST /api/payments/verify` recomputes the signature and the sale is marked paid only if it matches; a tampered signature fails. The interface labels development mode clearly so that it is never mistaken for a real charge.

4.8 Location Privacy

Nearby discovery is off by default and is switched on from the profile page. The server stores the position only for opted-in users, derives an approximate geohash [11] for display, and returns distances rounded to values such as "about 1.2 km away". Exact latitude and longitude are removed from every public serializer. Stored positions expire after a configurable period. A meeting point entered for a loan is returned only after both the borrower and the lender have agreed to it.

4.9 Trust Score and Moderation

A trust score between 0 and 100 is recomputed from completed transactions, on-time returns, overdue loans, the average review rating, account age and confirmed reports or high-risk moderation items, and is shown to other users as a star rating (score divided by 20). The administrator page lists held listings with their risk score and detected signals, and an approval or rejection updates both the moderation record and the listing.

4.10 ML Service

The service exposes `POST /ai/price-predict` and `POST /ai/fraud-score`. Training is reproducible: a generator script writes seed `listings.csv` (280 listings), and a training script splits it 70/15/15 with a fixed random seed, fits a Random Forest with 180 trees, maximum depth 12 and minimum leaf size 2, and writes the model and its metrics to disk. The Isolation Forest (200 trees, contamination 0.04) is fitted on 400 typical feature vectors plus a few planted anomalies such as extreme prices combined with bursts of duplicate listings.

4.11 Testing and Results

4.11.1 Automated and manual testing

The API has an automated suite written with the Node test runner (`npm run test:api`). Table 4.4 lists the behaviours it covers. In addition, the repository contains a nine-step manual checklist that walks through registration, browsing, selling with an AI price range, nearby discovery, a full borrow-and-return, a purchase with an invalid signature, fraud review, and the unauthorized-access cases.

Table 4.4. Behaviours verified by the automated API tests.

Area	Behaviour verified
Health	Service reports database state and payment mode.
Identity	Non-college e-mail rejected; Gmail password sign-up redirected to Google; missing Google token rejected; OAuth start needs credentials.
OTP	Register, verify and log in; expired OTP is refused.
Listings	Create, read, update, delete and search as a verified student; only the owner may modify a listing.
Location	Opt-in nearby search; exact coordinates never returned.
Loans	Request, approve, return and confirm a borrow.
Payments	Order creation, verification, and rejection of an invalid signature.
AI	Price and fraud endpoints do not crash the API, also when the ML service is unavailable.
Trust	Trust-score endpoint returns a bounded score.

4.11.2 Model performance

Table 4.5 reports the metrics written by the training script. The mean absolute error (MAE), root mean squared error (RMSE) and coefficient of determination (R^2) are computed on the validation and test splits of the synthetic data set.

Table 4.5. Random Forest price model: validation and test metrics ($n_{\text{train}} = 196$, $n_{\text{val}} = 42$, $n_{\text{test}} = 42$).

Split	MAE (Rs.)	RMSE (Rs.)	R^2
Validation	721.04	1723.94	0.953
Test	991.68	3932.21	0.623

4.11.3 Analysis

The validation R^2 of 0.953 and the test R^2 of 0.623 differ markedly. With only 42 rows per split and a price range that spans several orders of magnitude (a lent book versus a sold cycle), a few expensive items dominate the squared errors, which explains why the test RMSE is more than twice the validation RMSE while the MAE rises by about a third. The bounded blend with the formula price limits the damage a single poor tree-based prediction can do. Most importantly, the data are synthetic, so these numbers show that the pipeline trains, serializes, serves and reports honestly; they do not measure real campus prices.

4.12 Challenges and Resolutions

Table 4.6. Main problems met during development and how they were resolved.

Problem	Resolution
The app was a frontend over mock data with no persistence.	Added the Express and Mongoose server, the data models and REST routes, and connected every page through `src/api.js`.
MongoDB was not installed on the development machine.	Added an automatic fallback to an in-memory MongoDB server; the real connection string is a one-line configuration change.
Icon components cannot be stored in the database.	Stored an `iconKey` string and re-hydrated it to a component in the client.
Cross-origin errors between the Vite and Express ports.	Enabled CORS on the API and proxied `/api` through Vite.
An empty database made the interface look broken.	Wrote a seed script and seed automatically when the collections are empty.
Publishing and chat only changed React state or showed a toast.	Persisted listings, messages and transactions through POST endpoints and loaded them back on each page.
Prices and fraud signals needed intelligence beyond fixed rules.	Added the FastAPI service with two trained models and a graceful fallback when it is offline.

Chapter 5

Conclusion

5.1 Summary of Work

Campus Marketplace was designed and implemented as a three-tier system: a React interface, an Express and MongoDB API, and a FastAPI machine-learning service. All six objectives of Section 1.2 were addressed. Access is limited to verified college identities; the platform supports buying, borrowing, selling and lending with a complete loan life cycle; sellers receive an explainable price range; new listings are screened for fraud and risky ones are held for review; users carry a trust score while exact locations stay private; and the three tiers are independently runnable and covered by an automated API test suite.

5.2 Key Findings

- **Iterative delivery reduced risk.** Building the interface first on mock data and then adding backend, identity, marketplace logic and ML in separate increments kept each step testable and made integration problems visible early.
- **Graceful degradation matters.** Falling back to an in-memory database and continuing to accept listings when the ML service is offline kept the system demonstrable under imperfect conditions.
- **Blending a transparent formula with a learned model is safer than using the model alone on a small data set:** it bounds outliers and yields an explanation the user can read.
- **Honest evaluation is necessary.** The gap between validation ($R^2 = 0.953$) and test ($R^2 = 0.623$) performance shows that results on a small synthetic set must not be over-interpreted.

5.3 Limitations

The models are trained on synthetic seed data rather than real campus trades. Payments run in development mode unless Razorpay credentials are provided, and verification e-mail delivery depends on SMTP configuration. Messaging is request-based rather than real-time, and images are stored as references rather than processed or moderated.

5.4 Future Work and Recommendations

1. Collect anonymised, consented transaction data (the project already marks documents as exportable for training) and retrain the price and fraud models on real behaviour.
2. Move to live Razorpay credentials and add refunds and dispute handling.
3. Add real-time chat and push or e-mail reminders for due dates.
4. Deploy the three tiers with a managed MongoDB cluster, continuous integration, and monitoring of model drift.
5. Add image-based checks and richer explanations of fraud signals in the administrator interface.
6. Evaluate usability with a pilot group of students and feed the results into the next iteration.

5.5 Final Remarks

The project shows that a small team can combine established web engineering practice with lightweight machine learning to serve a concrete community need. Beyond the working system, it gave practical experience in requirements analysis, UML modelling, REST design, security-conscious authentication, payment verification, privacy by design and the honest reporting of model performance.

Bibliography

[1] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. In *Proc. IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008.

[3] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[4] C. Larman and V. R. Basili. Iterative and incremental developments: a brief history. *IEEE Computer*, 36(6):47–56, 2003.

[5] Object Management Group. *Unified Modeling Language (UML), Version 2.5.1*. OMG, 2017.

[6] I. Sommerville. *Software Engineering*, 10th edition. Pearson, 2016.

[7] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[8] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.

[9] D. Hardt (ed.). The OAuth 2.0 Authorization Framework. RFC 6749, IETF, 2012.

[10] H. Krawczyk, M. Bellare, and R. Canetti. HMAC: Keyed-hashing for message authentication. RFC 2104, IETF, 1997.

[11] R. W. Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.

[12] Meta Open Source. React Documentation. `https://react.dev`.

[13] OpenJS Foundation. Express – Node.js web application framework. `https://expressjs.com`.

[14] MongoDB, Inc. MongoDB Documentation. `https://www.mongodb.com/docs`.

[15] S. Ramírez. FastAPI Documentation. `https://fastapi.tiangolo.com`.